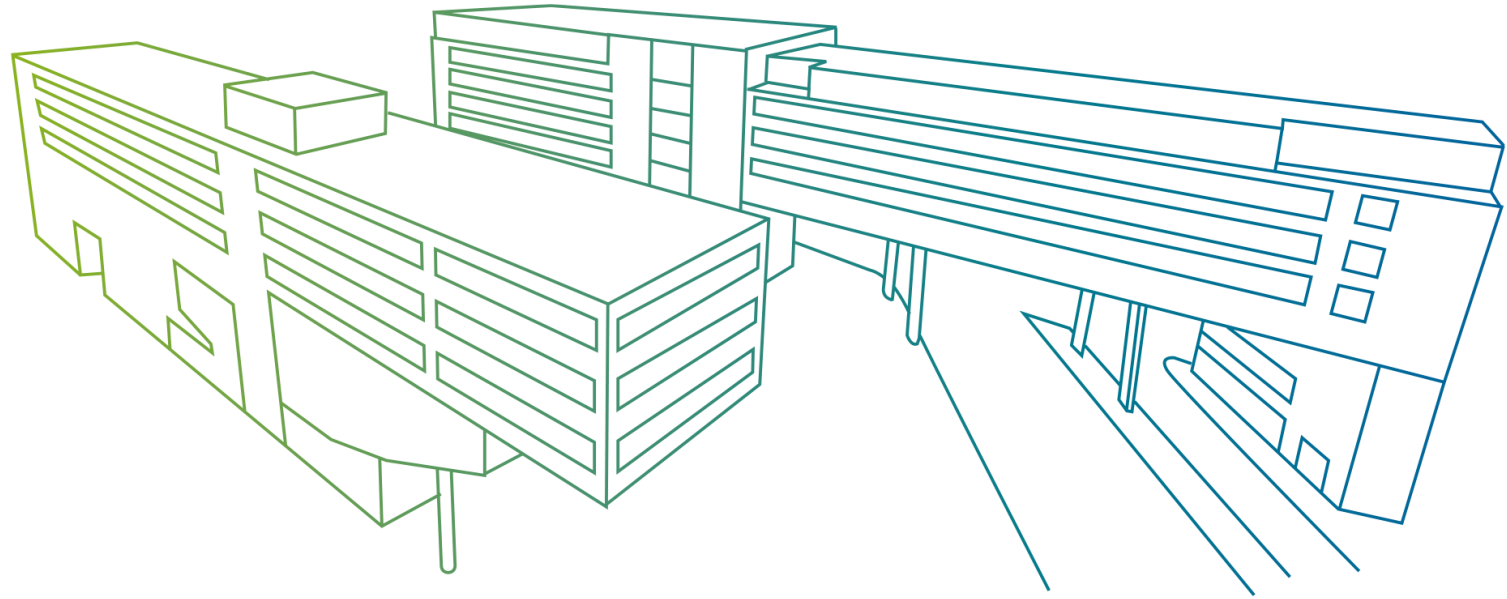


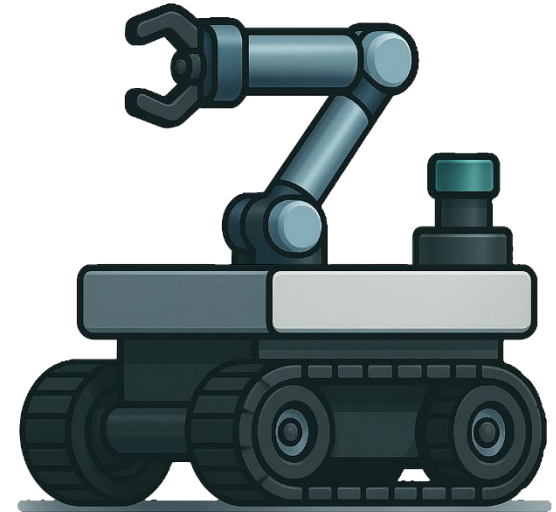


Probabilistic Robotics

Basic Introduction
Lucas Muster, MSc

Introducing Probabilistic Robotics

- Organisation of this course
- PRO and PRO Lab
- Some definitions
- Motivation: Why do we use statistics?
- Notation: Vectors, matrices and distributions
- Basic math understandings



Organisation of this course

- 100% Theory Exam
 - Practical implementation will be in the Lab
- Open Questions
- No learning by heart - understanding the algorithm is tested
- Grading Scale:
 - Excellent (Grade 1): 90–100%
 - Good (Grade 2): 80–89%
 - Satisfactory (Grade 3): 70–79%
 - Sufficient (Grade 4): 60–69%
 - Insufficient (Grade 5): Below 60% in either component → repetition required

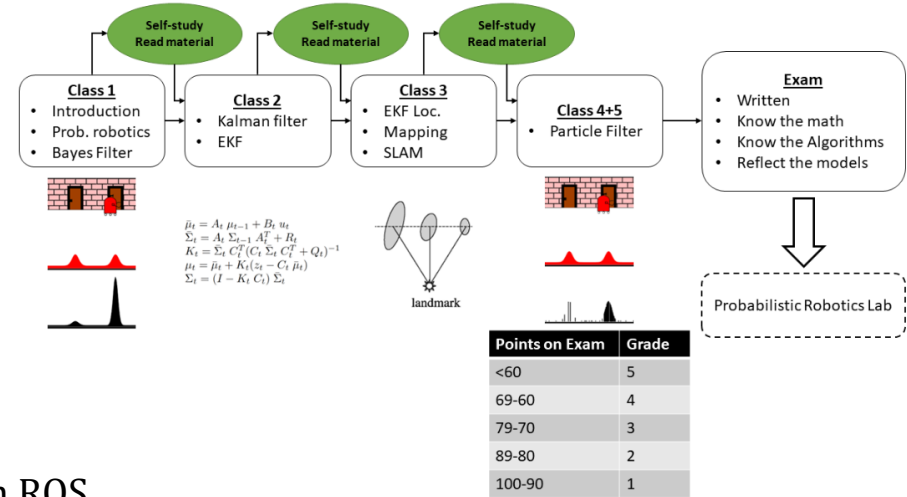
PRO and PRO Lab

- Probabilistic Robotics

- 5 classes
- Exam
- Preparation for the Lab

- Probabilistic Robotics Lab

- Implementation of localization algorithms in ROS
- Project submission

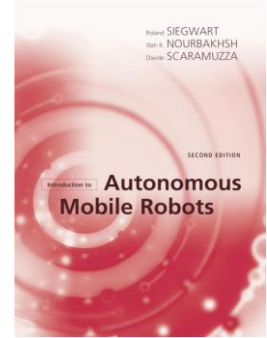


ROS 2™

Literature

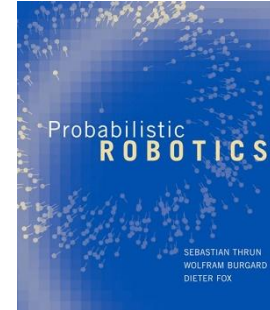
- Introduction to Autonomous Mobile Robots

Roland Siegwart, Illah Nourbakhsh, and Davide Scaramuzza



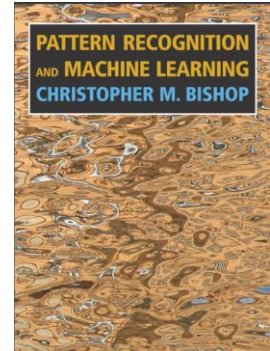
- Probabilistic Robotics

Sebastian Thrun, Wolfram Burgard, and Dieter Fox



- Pattern Recognition and Machine Learning

Christopher M. Bishop



Structure of a Mobile Robot: Software

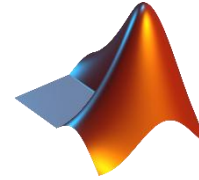
- How to program these robots?
 - C/C++: Control in real-time
 - Python: Modelling
 - Matlab: Toolboxes
 - ROS and ROS2: Robot operating system
 - Easy communication
 - Ready-to-use software
 - Docker: Containerized Setup
 - NVIDIA: GPU acceleration
 - Deep Learning
 - Auton. driving toolbox



Picture Source: <https://isocpp.org/>



Picture Source: <https://www.python.org/>



Picture Source: <https://de.mathworks.com/products/matlab.html>



Picture Source: <https://www.ros.org/>



Picture Source: <https://www.nvidia.com/de-at/>



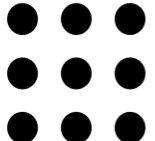
Picture Source: <https://www.docker.com/>

→ Again: Depending on the application of the mobile robot

Introduction to ROS2



- Robot Operating System (ROS) is a flexible and open-source framework for developing robot application and systems.
- Defines components, interfaces and tools for building advanced robots.
- Standard software platform for developers.
- It plays a pivotal role in advancing robotics by simplifying development, fostering collaboration, and enabling rapid prototyping and testing.

 ROS 2TM

Picture Source: <https://www.ros.org/>

Why do we use ROS?

- Fast prototyping in simulated worlds
- Open source packages
- Abstract Communication
 - Nodes
 - Messages
- Multi-Language Support
 - C/C++, Python, ...
 - Matlab, R
- Debugging and visualization tools

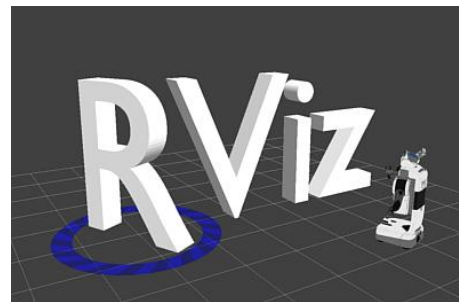


GAZEBO

<https://gazebo-sim.org/home>



<https://moveit.picknik.ai/main/index.html>



<http://wiki.ros.org/rviz>

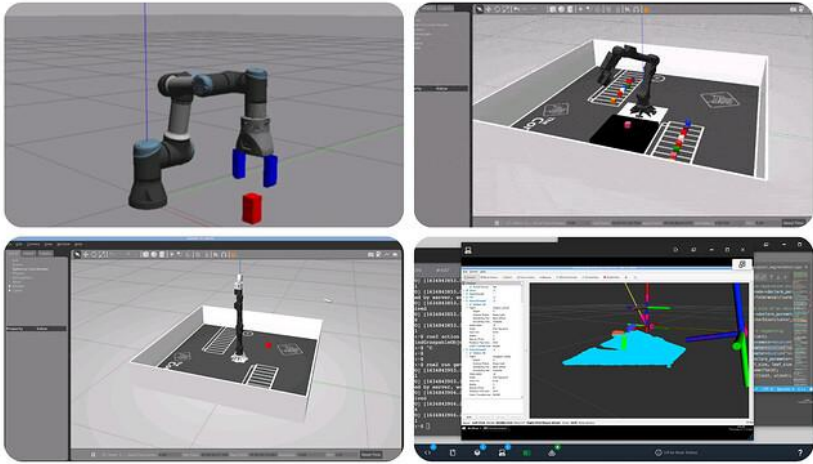
Motivation

- Our Robot stands at $\vec{x}_t = (x_t \ y_t \ \theta_t)^T$
 - Two wheels move – where am I after Δt ?
- We will drive in circular pattern

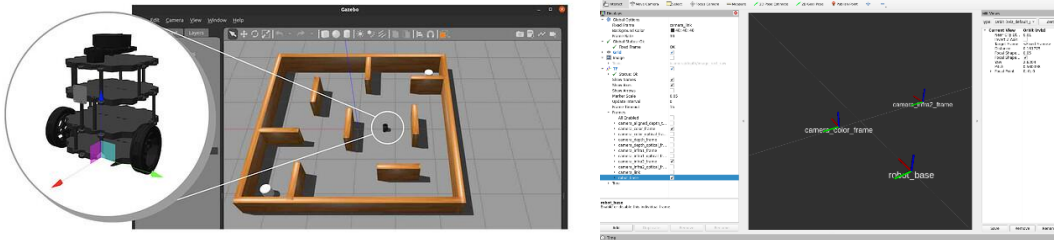


- Yes, there is software for mobile robot localization (it is ROS), but...
 - Different to industrial robotics – we integrate over time (in industrial robotics, we have absolute values)
 - There is noise → we must use sensor fusion to minimize the influence of noise
 - This leads to machine learning
- Mobile robotics part: Learning the robot pose from data & reasoning over time
 - This leads to the Bayes filter
 - We focus on particle filters

Why do we use ROS?

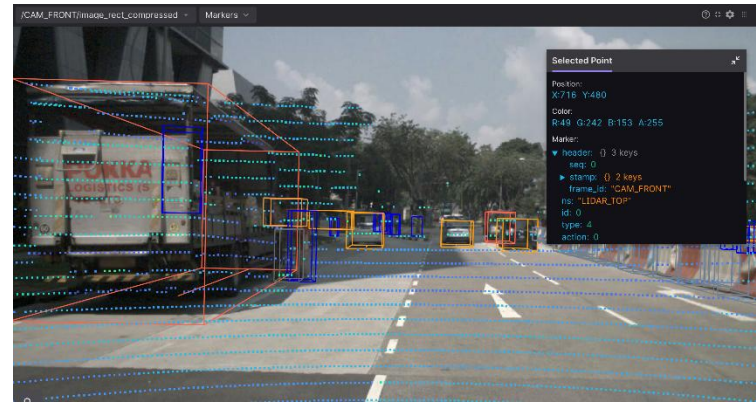


<https://discourse.ros.org/t/new-ros-online-courses-gazebo-simulator-intermediate-ros2-1f-ros2/26774>

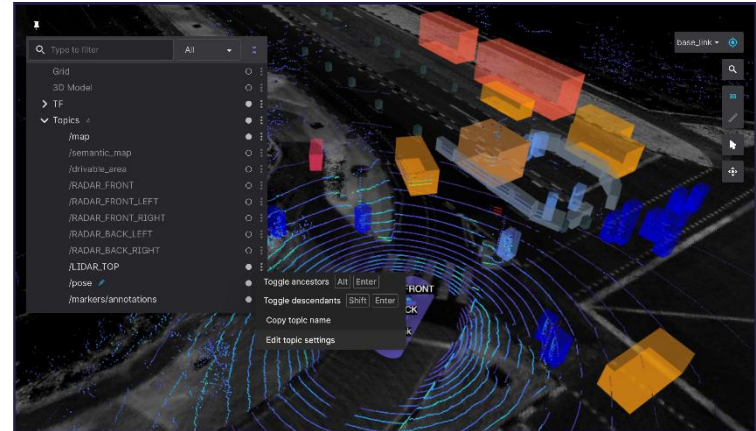


<https://discourse.ros.org/t/new-ros-online-courses-gazebo-simulator-intermediate-ros2-1f-ros2/26774>

<https://github.com/IntelRealSense/realsense-ros>



<https://docs.ros.org/en/foxy/How-To-Guides/Visualizing-ROS-2-Data-With-Foxglove-Studio.html>

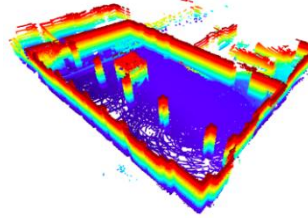


<https://docs.ros.org/en/jazzy/How-To-Guides/Visualizing-ROS-2-Data-With-Foxglove-Studio.html>

Motivation

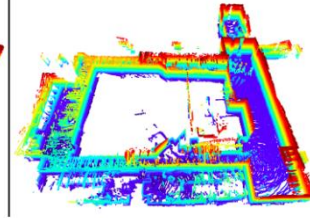


a) Scenario 1



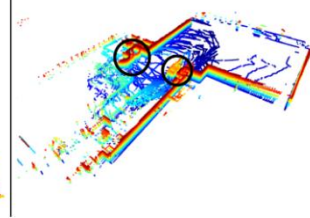
25m

b) Scenario 2



25m

c) Scenario 3



25m

Height [m]
4.5
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0

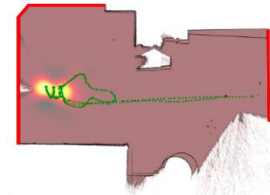
a) Scenario 1



25m

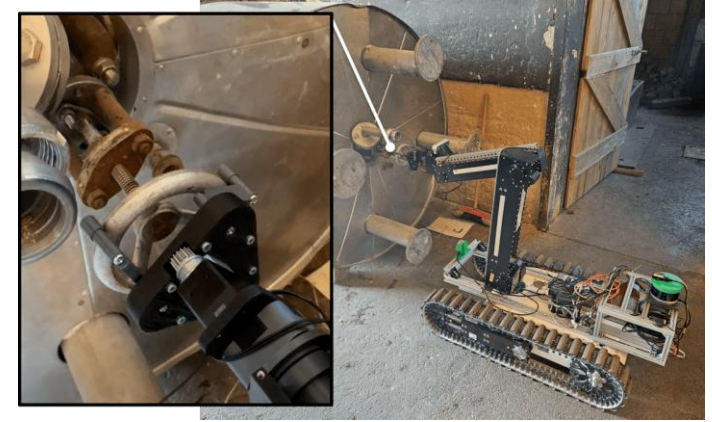
Radiation [$\mu\text{Sv/h}$]
6.73
5.43
4.08
2.72
1.37
0.01

b) Scenario 3



25m

Radiation [$\mu\text{Sv/h}$]
1.68
1.34
1.01
0.67
0.34
0.00



Some Definitions: Sensors



- Analysis of the environment
- Infer own motion
- Reasoning is necessary (sensor data to physical interpretation)
- Fusion of information sources
 - Why? Minimize impact of noise



Figure 4: From left to right. Sick 3D Laserscanner, Intel Realsense, Olive IMU, Nema 34 Stepper Motor with encoder [2]

Some Definitions: Perception

- Processing of sensor data
- External sensors
 - Usage of external information from environment
 - 2D/3D Laser Scanner
 - Imaging Sensor (RGB, NIR or TIR Camera)
 - Etc...



Figure 5: Example of external sensors. Left: Sick 3D Laserscanner, Right: Intel Realsense [3]

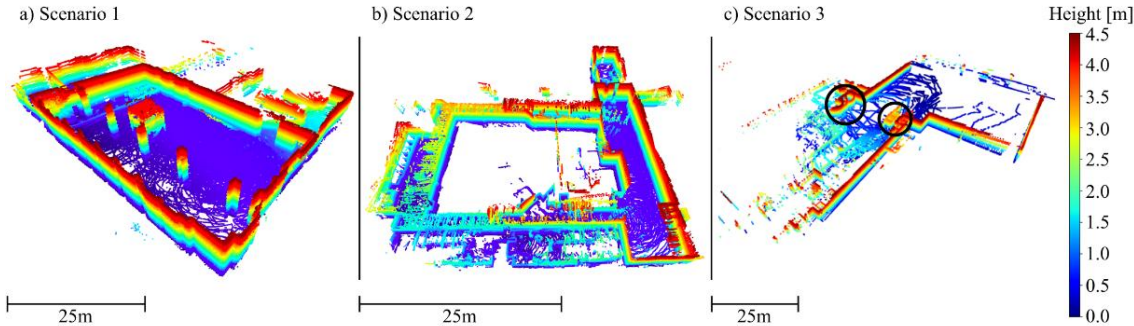


Figure 6: 3D Mapping of the Tritolwerk in Lower Austria (a) and the Atomic Power Plant Zwentendorf (b) and (c) (Schwaiger, 2024)

Some Definitions: Perception

- Processing of sensor data
- Internal Sensors
 - Measuring Information “inside” the robot
 - Odometry (wheel motion to robot motion)
 - IMU (Inertial Measurement Unit)
 - Etc...



Figure 8: Example of internal sensors. Left: Olive IMU, Right: Nema 34 Stepper Motor with encoder [3]

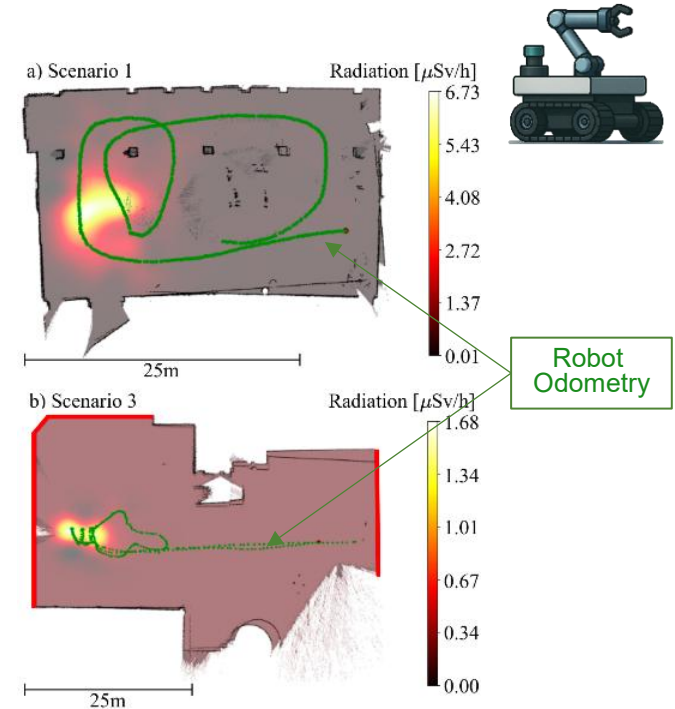


Figure 7: Visualization of Robot Movement (Odometry) around radioactive source at the Tritolwerk in Lower Austria (top) and the Atomic Power Plant Zwentendorf (bottom) (Schwaiger, 2024)

Some Definitions: Mapping



- Creates a 2D/3D representation of the world using sensor data
- This map is then used for localization and planning
- Current research topic: Simultaneous localization and mapping (SLAM)
 - Fuse Mapping and Localization

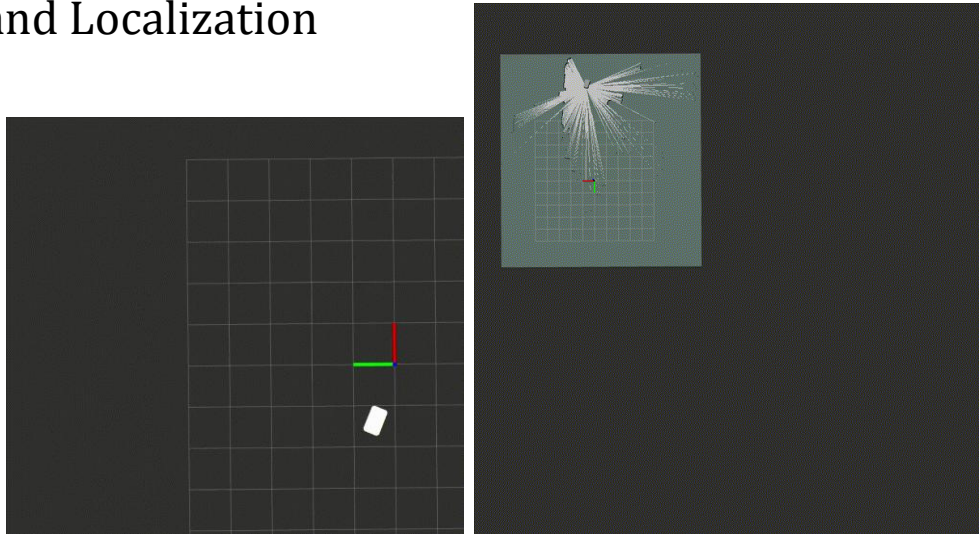


Figure 9: Mapping Example with ROS2 and Nav2 (Macenski, 2023)

Some Definitions: Planning

- Plan motion and path to achieve "goal"
- A path is a sequence of poses
- We need models for planning
 - Motion models (Kinematics)
 - Sensor models (Interpretation of sensor data)

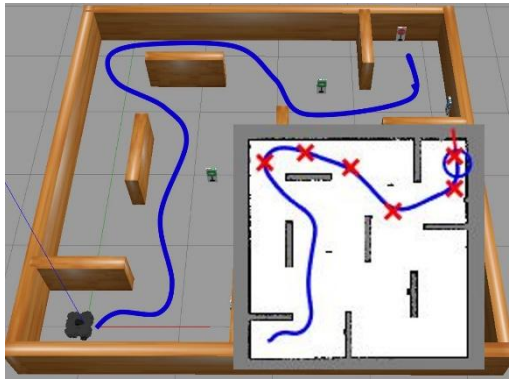
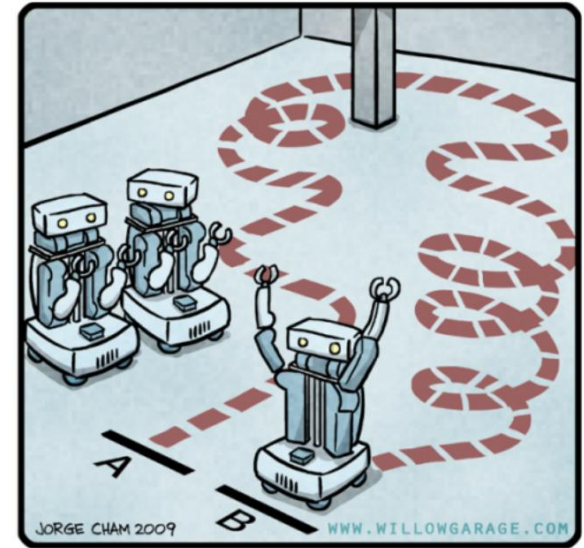


Figure 10: Matlab example for Autonomous Navigation for Mobile Robots and UGV [3]



R.O.B.O.T. Comics



"HIS PATH-PLANNING MAY BE
SUB-OPTIMAL, BUT IT'S GOT FLAIR."

Picture Source: <http://wiki.ros.org/navigation>

Sensor Technology: Odometry



- Increment motion over time
- Typically based on angular sensors
- Based on wheel motion, we can calculate robot motion

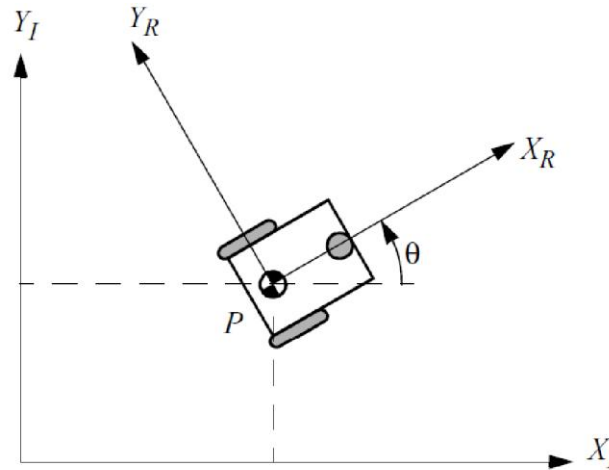


Figure 13: The new pose is based on the wheel motion and a robot model (Siegwart, 2004).
The pose of the robot is described in a global frame.

Sensor Technology: IMU



- Inertial Measurement Unit
 - Linear acceleration and angular velocity
 - Often as a quaternion-based representations
 - Connected to a specific frame (typically, we must transform it)

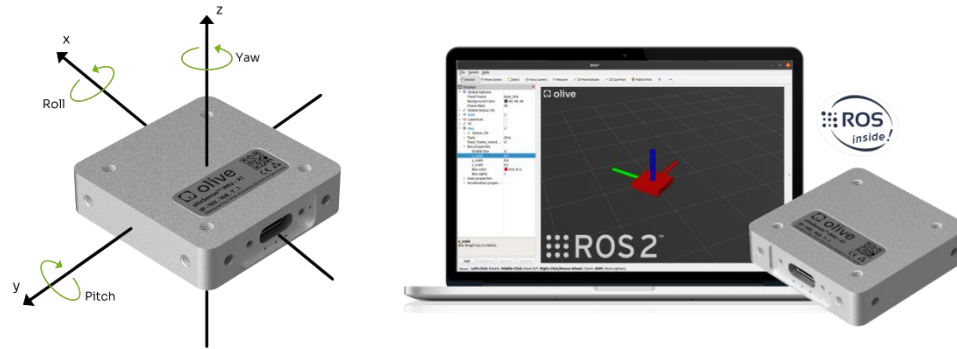


Figure 14: Olive olixSense IMU [5]

Sensor Technology: 1D Distance Sensor

- Sensor principle: Ultrasonic or Infra-red
- Note: Reflections and interferences are a huge problem
- Limited in range

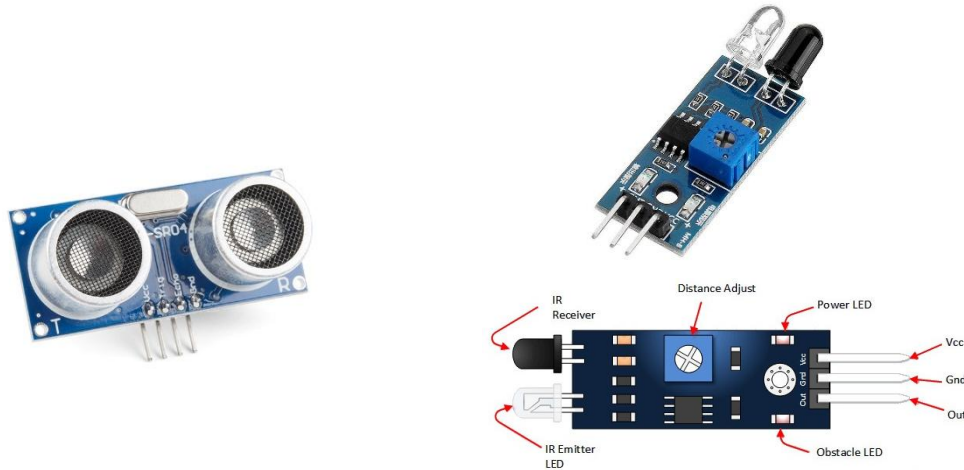


Figure 15: Left: Ultrasonic Distance Sensor, Right: IR Sensor [6]

Sensor Technology: Laserscanner

- Typically, the default sensor
- 2D or 3D
- Interpretation: Distance measure including angle
- Fog and glass are real-life problems

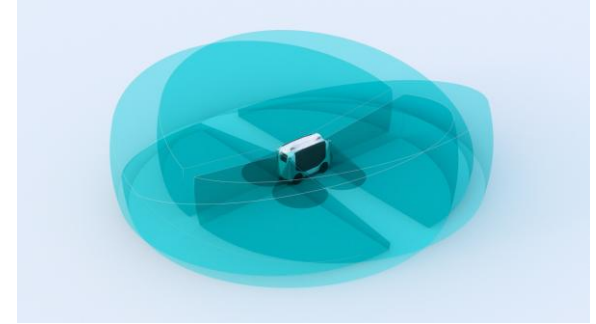
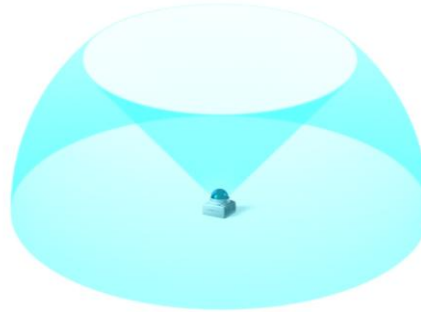
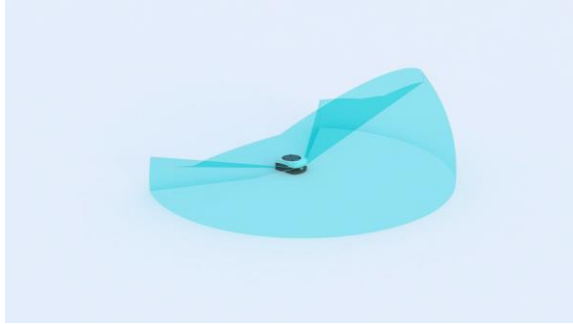


Figure 16: 3D Laserscanner Sick (top) and the Livox MID-360 Laserscanner (bottom) [7]

Sensor Technology: Camera



- Available in different wavelengths: visible or near/far infra red
- Based on machine vision and learning, robots can "see"
- Different types of cameras
 - Stereo Camera
 - Depth Camera

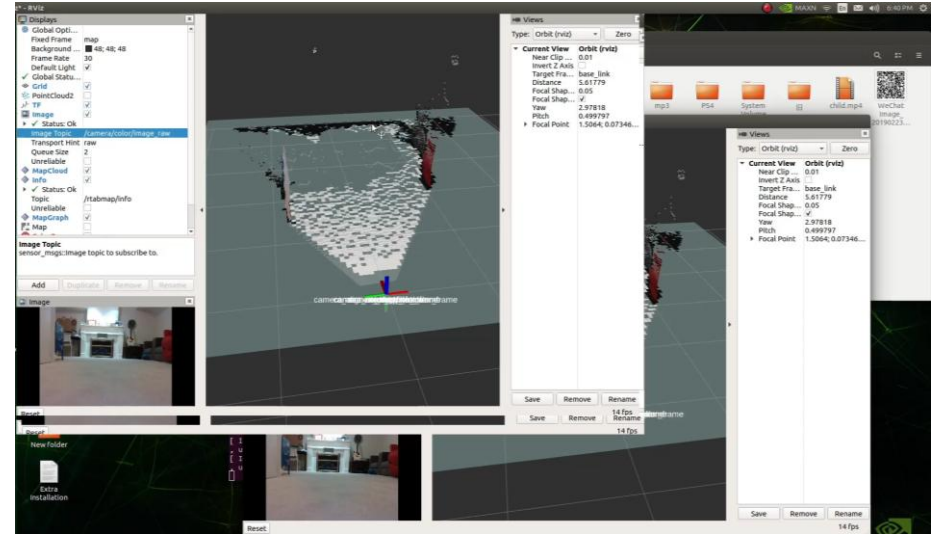
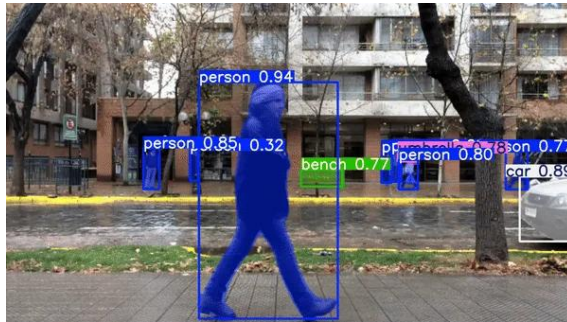


Figure 17: Left: Object detector using Yolo11, Right: ROS rtmap_ros Package [8]

Sensor Technology: GPS/GNSS

- GNSS = GPS, Galileo, GLONASS, BeiDou
- Provides **global positioning** for outdoor navigation
- Used for **waypoint tracking**, **SLAM**, and **global localization**
- **Limitations:** low rate, signal loss, multipath effects



Figure 18: LORD Sensing 3DM-GX5 (IMU + GNSS) [9]

Motivation

- Traditional methods are restricted
 - Real environment is hard to formalize
 - Interaction with humans → perception/understanding
 - For full automation → intelligent systems are needed
 - How to decide in uncertainty?
- But,...
 - do we want intelligent robots?
 - where do we need AI-driven robots?
 - can we trust these machines? (see clever Hans)
 - can we generate robots for daily life? (see no-free-lunch theorem)

Some Definitions

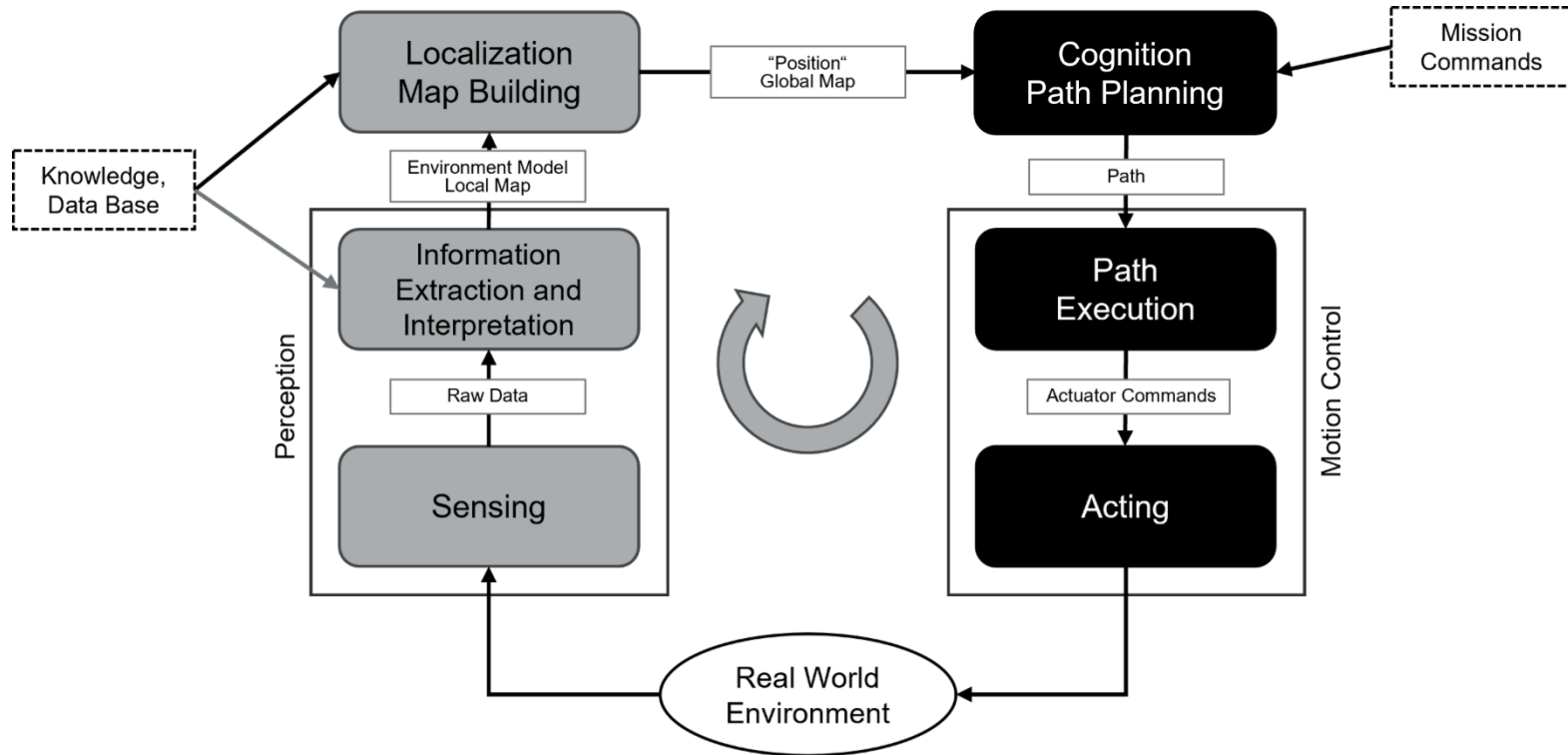


Figure 3: "Control loop" for mobile robotics (Siegwart,2004)

Dealing with Disturbances



Every real-world mobile robot is subject to uncertainties — both external disturbances and internal sensor noise. A robust control system must detect and reject them.

Disturbances are inevitable.

- Mobile robots operate in unpredictable environments:
 - Slippery floors, inclines, moving obstacles, wind (for drones)
- These affect the plant (the physical system)



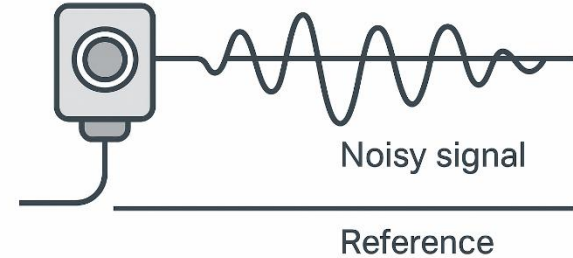
Dealing with Disturbances



Noise corrupts the measurements.

- Sensor readings are never perfect:
 - LiDAR returns noisy ranges
 - Encoders have resolution limits
 - IMUs drift over time
- This affects the feedback in your control loop

SENSOR NOISE



Without compensation, your control system will degrade.

- You may follow the wrong path
- Oscillations may arise
- The system may become unstable

Motivation: Sources of Error and Uncertainty

Assumption: Mobile robot using differential drive

We can calculate movement using:

- Dimensions of the robot
- Information concerning engine
- Control theory software

Challenges: In most applications, the calculated position will not match the actual position due to:

- Slippage
- Errors in production
- Sensor resolution limitations

Motivation: Overcoming Uncertainty

- How can we overcome errors?
 - **Noisy data:** Sensors provide imperfect data with noise.
 - **Unknown errors:** Certain errors are unpredictable and can't be modeled deterministically.
- **Solution:**
 - Use **Statistical Inference** to handle noise and errors.
 - Treat all measurements and states as **random variables**, allowing probabilistic models to make better estimations.

Motivation: Need for Probabilistic Models

- **The Need for Probabilistic Methods:**

- **Deterministic models** are insufficient when dealing with real-world uncertainties.
- Probabilistic methods allow for a more **robust and adaptable** approach to state estimation and control.

- **Why Probabilistic Models?:**

- They allow robots to handle **sensor inaccuracies, actuator uncertainties, and unpredictable environments**.
- **Example:** A mobile robot navigating with noisy GPS or LiDAR data.

Motivation: Key Advantages of Probabilistic Approaches

Improved Accuracy:

By accounting for noise and errors, probabilistic methods offer more accurate state estimations than deterministic methods.

Resilience to Uncertainty:

Probabilistic models and algorithms help robots make better decisions even in highly uncertain or dynamic environments.

Examples in Practice:

- **Autonomous vehicles** use probabilistic algorithms (e.g., Kalman Filters, Particle Filters) for localization and obstacle detection.
- **Mobile robots** use probabilistic models for mapping and navigation.

Motivation: Key Advantages of Probabilistic Approaches

Aspect	Deterministic	Probabilistic
Output	Single value	Probability distribution
Noise handling	Ignored or fixed	Explicitly modeled
Repeatability	Always identical	Can vary
Realism	Idealized	Real-world aware
Example	$x = f(u)$	$p(x \mid z, u)$

Notation: State, Measurement, and Control

- State at time t

$$\vec{x}_t = \begin{bmatrix} x_t \\ y_t \\ \theta_t \end{bmatrix}$$

Position (x_t, y_t) and orientation (θ_t) of the robot.

- Measurement at time t

$$\vec{z}_t = \begin{bmatrix} z_1 \\ z_2 \\ \dots \end{bmatrix}$$

Sensor data (e.g., from cameras, LiDAR, GPS).

Notation: State, Measurement, and Control

- Control at time t :

$$\overrightarrow{u}_t = \begin{bmatrix} v \\ \omega \end{bmatrix}$$

Example: v : Linear velocity, ω : Angular velocity.

- Time series:

$$\overrightarrow{x_{0:t}} = \{x_0, x_1, \dots, x_t\}$$

Represents the trajectory of the robot over time.

Attention: YOU have to design the robot state $\overrightarrow{x}_t$.

Notation: Random Variables and Probability Distributions

Initial situation: Random Variable X

Describes uncertain variables like sensor measurements or robot state.

Example:

Coin flip X with $x = \{head, tail\}$ or robot's pose X with $x_t = \{x_t, y_t, \theta_t\}$.

Assumption: All random variables follow a probability density function

Notation: Random Variables and Probability Distributions

Example: Coin flip X with $x = \{head, tail\}$

Fair coin:

- Probability of "Heads" is $p=0.5$ (or 50%).
- Probability of "Tail" is $p=0.5$ (or 50%).

Notation: Random Variables and Probability Distributions

Example: Coin flip X with $x = \{head, tail\}$

Fair coin:

- Probability of "Heads" is $p=0.5$ (or 50%).
 - Probability of "Tail" is $p=0.5$ (or 50%).
-
- Biased coin:
 - Probability of "Heads" is $p=0.7$ (or 70%).
 - Probability of "Tail" is $p=0.3$ (or 30%).

Notation: Random Variables and Probability Distributions

Example: Coin flip X with $x = \{head, tail\}$

Fair coin:

- Probability of "Heads" is $p=0.5$ (or 50%).
- Probability of "Tail" is $p=0.5$ (or 50%).



$$E(X) = p = 0.5$$

$$\text{Var}(X) = p(1-p) = 0.25$$

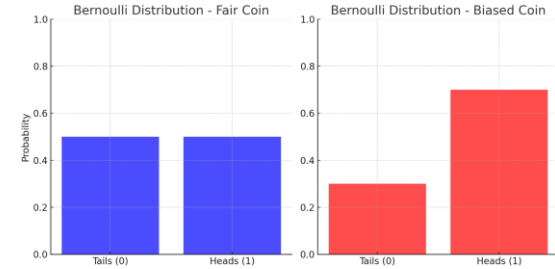
• Biased coin:

- Probability of "Heads" is $p=0.7$ (or 70%).
- Probability of "Tail" is $p=0.3$ (or 30%).



$$E(X) = p = 0.7$$

$$\text{Var}(X) = p(1-p) = 0.21$$



Notation: Discrete & Continuous Probability Distributions

Binomial Distribution: Used for discrete events like coin flips.

$$P(x) = \binom{n}{k} p^k (1 - p)^{n-k}$$

- n = total number of trials
- k = number of successes
- p = probability of success on a single trial
(fair coin: 0.5, biased coin: 0.7)

Example: $n = 10$

Notation: Discrete & Continuous Probability Distributions

$P(X = k)$...probability of getting exactly k success in n trials

$$\begin{aligned} P(X = 3) &= \binom{n}{k} * p^k * (1 - p)^{n-k} = \binom{10}{3} * 0,5^3 * (1 - 0,5)^{10-3} = \\ &= \frac{10!}{3! (10 - 3)!} * 0,5^3 * (1 - 0,5)^{10-3} = 11,7\% \end{aligned}$$

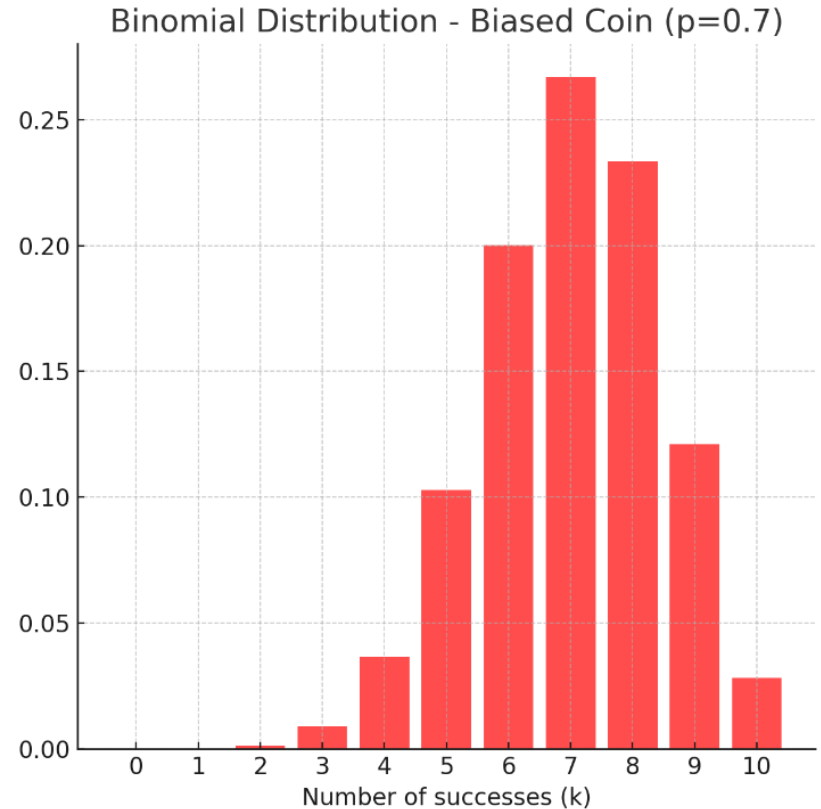
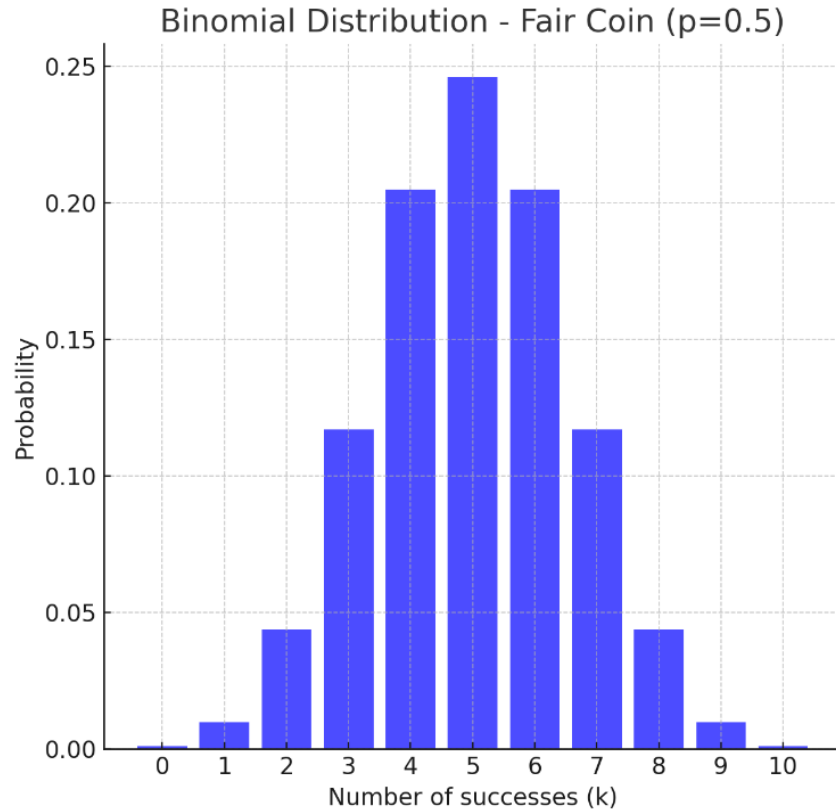
Fair coin:

- $E(X) = n * p = 10 * 0,5 = 5$
- $\text{Var}(X) = n * p * (1 - p) = 10 * 0,5 * (1 - 0,5) = 2,5$

Biased coin:

- $E(X) = n * p = 10 * 0,7 = 7$
- $\text{Var}(X) = n * p * (1 - p) = 10 * 0,7 * (1 - 0,7) = 2,1$

Notation: Discrete & Continuous Probability Distributions



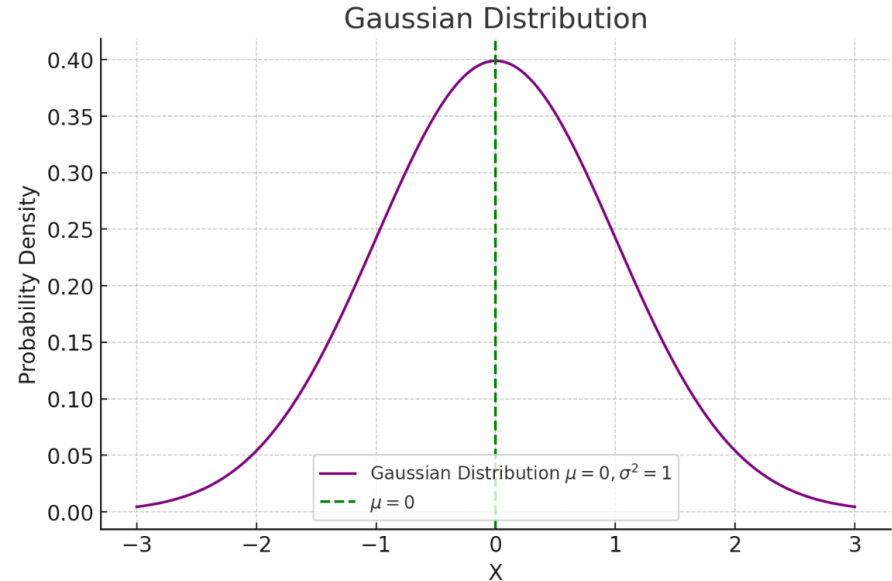
Notation: Discrete & Continuous Probability Distributions

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

Notation: Discrete & Continuous Probability Distributions

Gaussian Distribution: Describes continuous values with mean μ and variance σ^2 .

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$



Notation: Conditional Probability and Gaussian Variables

Using Random Variables for State Description:

- In probabilistic robotics, the state of the robot is modeled as a random variable:

$$P(X = \vec{x}) := p(\vec{x})$$

- This expresses the probability distribution over the possible states $\vec{x}$, considering the uncertainty in the robot's knowledge.

Notation: Conditional Probability and Gaussian Variables

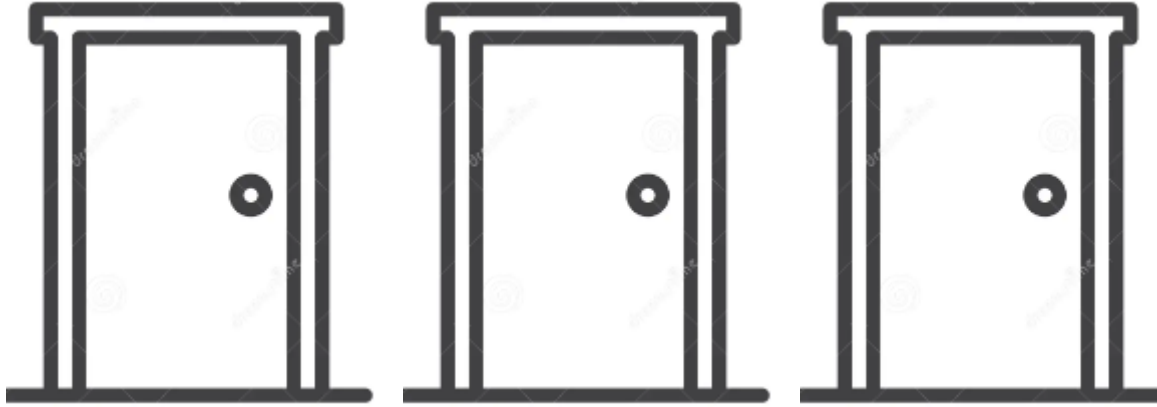
Conditional Probability:

- The probability of an event A occurring given that another event B is true:

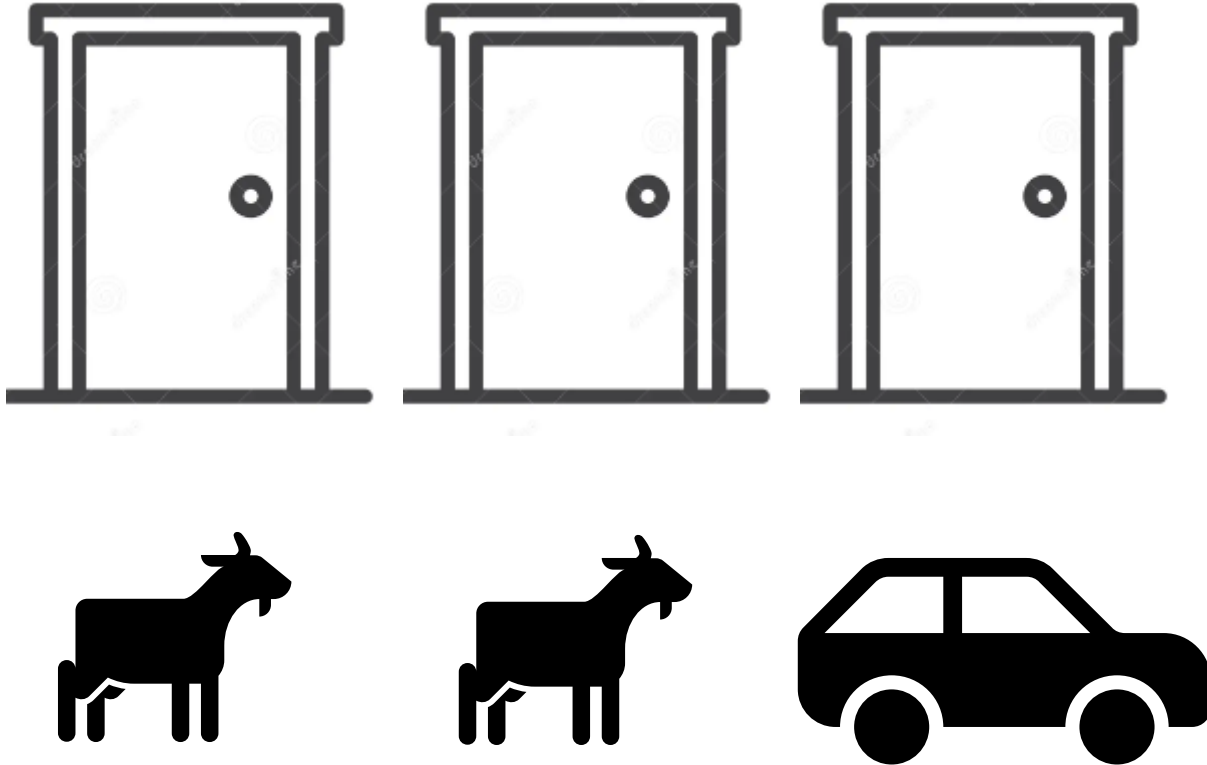
$$P(A | B) = \frac{P(B | A) * P(A)}{P(B)}$$

- $P(A | B)$: Probability of A given B (posterior)
- $P(B | A)$: Likelihood of B given A
- $P(A)$: Probability of A given B (posterior)
- $P(B)$: Normalizing constant (total probability of B)

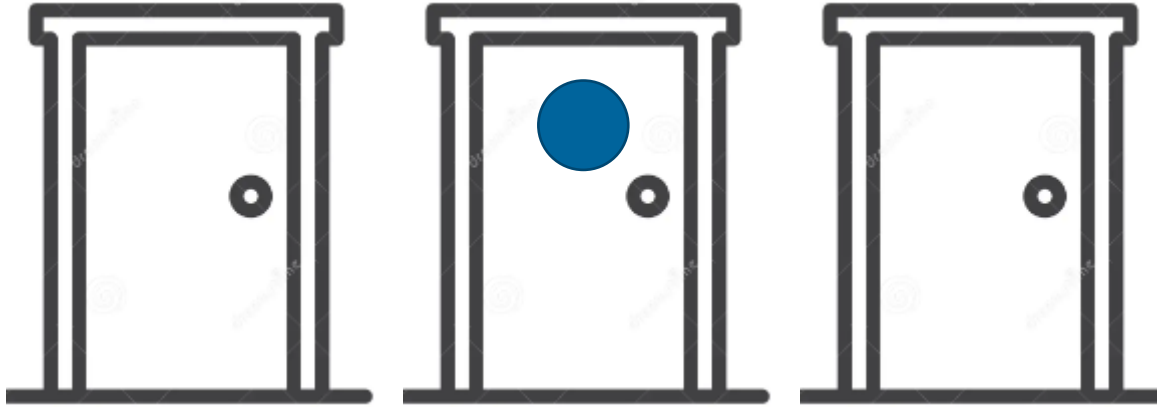
The Monty Hall Problem



The Monty Hall Problem

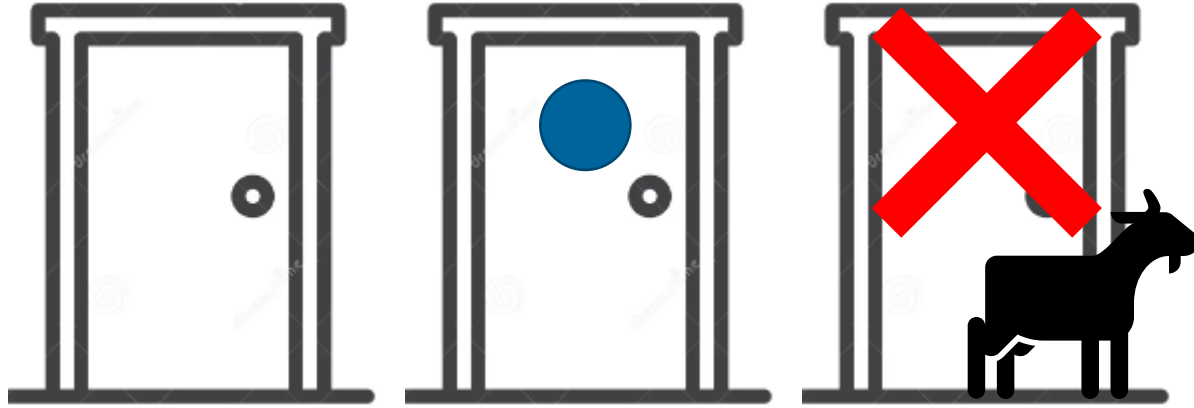


The Monty Hall Problem



- Winning probability after initial selection: $1/3$
- The probability of winning always sum up to 1
- Example: we are choosing door 2

The Monty Hall Problem

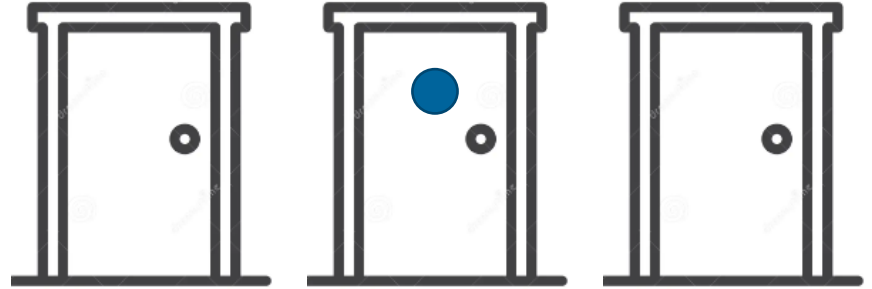


- One false door (with a goat) gets eliminated/opened.
- Should we change?
- What is the probability to win when we change?

The Monty Hall Problem

Bayes applied to Monty Hall

	door 1	door 2	door 3
$P(\text{door})$		$1/3$	

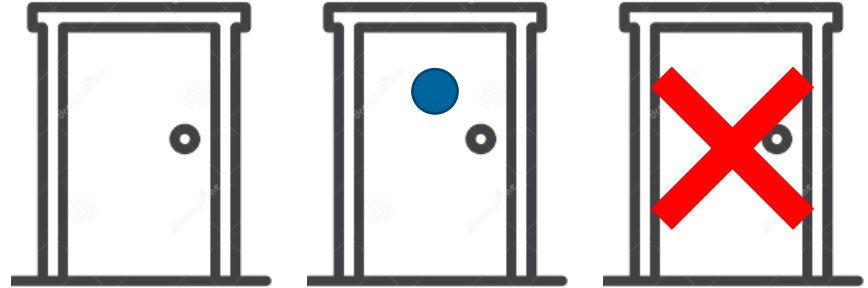


The Monty Hall Problem

Bayes applied to Monty Hall

	door 1	door 2	door 3
$P(\text{door})$	$1/3$		
$P(\text{obs.} \text{door})$	1	$1/2$	0

Door 3 is being opened!

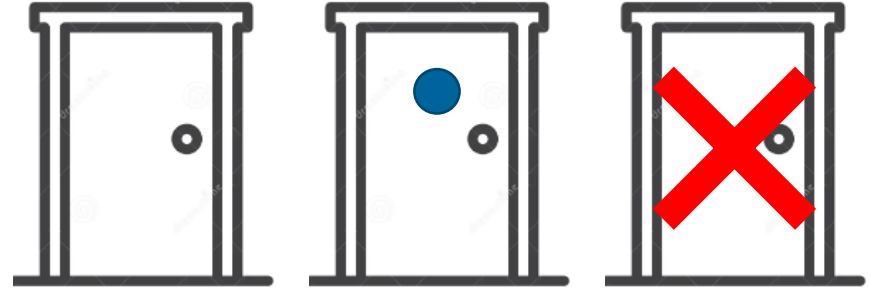


Multiply prior and likelihood: $P(\text{obs.}, \text{door}) = P(\text{door})P(\text{obs.}|\text{door})$

The Monty Hall Problem

Bayes applied to Monty Hall

	door 1	door 2	door 3
$P(\text{door})$	$1/3$		
$P(\text{obs.} \text{door})$	1	$1/2$	0
$P(\text{obs.}, \text{door})$	$1/3$	$1/6$	0



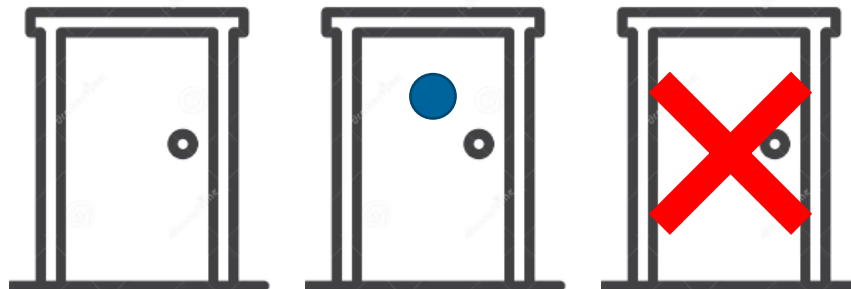
Multiply prior and likelihood: $P(\text{obs.}, \text{door}) = P(\text{door})P(\text{obs.}|\text{door})$

Calculate normalisation constant: $P(\text{obs.}) = \sum_{\text{door}} P(\text{obs.}, \text{door})$

The Monty Hall Problem

Bayes applied to Monty Hall

	door 1	door 2	door 3
$P(\text{door})$	$1/3$		
$P(\text{obs.} \text{door})$	1	$1/2$	0
$P(\text{obs.}, \text{door})$	$1/3$	$1/6$	0
$P(\text{obs.})$	$1/2$		
$P(\text{door} \text{obs})$	$2/3$	$1/3$	0



Multiply prior and likelihood: $P(\text{obs.}, \text{door}) = P(\text{door})P(\text{obs.}|\text{door})$

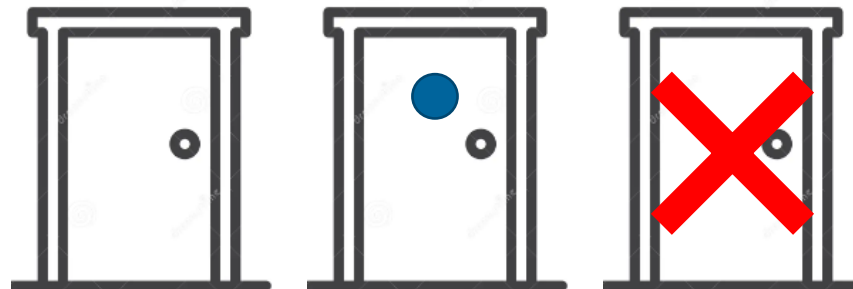
Calculate normalisation constant: $P(\text{obs.}) = \sum_{\text{door}} P(\text{obs.}, \text{door})$

Update belief with Bayes theorem: $P(\text{door}|\text{obs}) = P(\text{obs.}, \text{door})/P(\text{obs.})$

The Monty Hall Problem

Bayes applied to Monty Hall

	door 1	door 2	door 3
$P(\text{door})$	$1/3$		
$P(\text{obs.} \text{door})$	1	$1/2$	0
$P(\text{obs.}, \text{door})$	$1/3$	$1/6$	0
$P(\text{obs.})$	$1/2$		
$P(\text{door} \text{obs})$	$2/3$	$1/3$	0



Multiply prior and likelihood: $P(\text{obs.}, \text{door}) = P(\text{door})P(\text{obs.}|\text{door})$

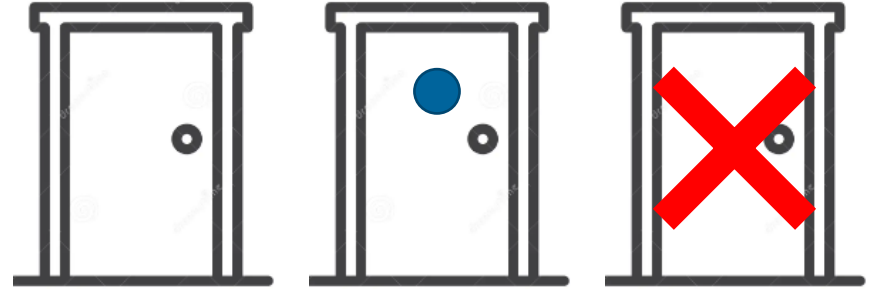
Calculate normalisation constant: $P(\text{obs.}) = \sum_{\text{door}} P(\text{obs.}, \text{door})$

Update belief with Bayes theorem: $P(\text{door}|\text{obs}) = P(\text{obs.}, \text{door})/P(\text{obs.})$

The Monty Hall Problem

Bayes applied to Monty Hall

	door 1	door 2	door 3
$P(\text{door})$	$1/3$		
$P(\text{obs.} \text{door})$	1	$1/2$	0
$P(\text{obs.}, \text{door})$	$1/3$	$1/6$	0
$P(\text{obs.})$	$1/2$		
$P(\text{door} \text{obs})$	$2/3$	$1/3$	0



Multiply prior and likelihood: $P(\text{obs.}, \text{door}) = P(\text{door})P(\text{obs.}|\text{door})$

Calculate normalisation constant: $P(\text{obs.}) = \sum_{\text{door}} P(\text{obs.}, \text{door})$

Update belief with Bayes theorem: $P(\text{door}|\text{obs}) = P(\text{obs.}, \text{door})/P(\text{obs.})$

→ Switching the door changes the probability of winning from $1/3$ to $2/3$.

Notation: Conditional Probability and Gaussian Variables

Normalization:

The constants α and η are used to ensure that probabilities sum to 1:

$$P(A | B) = \alpha * P(B | A) * P(A) \text{ or } P(A | B) = \eta * P(B | A) * P(A)$$

Combination of evidences:

$$p(a | b, c) = \frac{p(b | a, c) * p(a | c)}{p(b | c)}$$

Notation: Conditional Probability and Gaussian Variables

Gaussian (Normal) Distribution:

The Gaussian distribution is commonly used to represent uncertainty in continuous states, especially for sensor noise or motion models:

$$p(x) = \mathcal{N}(\vec{x} \mid \mu, \Sigma)$$

- μ ... mean vector (expected state)
- Σ ... Covariance matrix (spread of the distribution)

Notation: Conditional Probability and Gaussian Variables

Joint Probability of Two Variables:

The probability of two random variables A and B occurring together:

$$P(A, B) = P(A \wedge B)$$

This is useful in multi-sensor fusion, where different sensor measurements are combined probabilistically.

Basic Math: Reasoning Over Time

Bayes rule for mobile robots

$$p(\vec{x}_t | \vec{z}_t, \vec{u}_t) = \frac{p(\vec{z}_t | \vec{x}_t) * p(\vec{x}_t | \vec{u}_t)}{p(\vec{z}_t)}$$

Where is the robot?

Localization: The primary goal is to estimate the robot's current **pose** $\vec{x}_t$.

→ This is the core problem of **robot localization**.

Basic Math: Reasoning Over Time

Basis for the estimation: Motion commands and kinematics:

- The robot's motion is driven by a series of control inputs $\overrightarrow{u_t}$, which describe how the robot moves over time.
- The **motion model** predicts how the robot's position changes as it executes these commands.

Basic Math: Reasoning Over Time

Overcoming errors: Using additional sensor measurements:

- To refine the estimate and account for errors (e.g., actuator noise or slippage), we rely on **sensor measurements** $\vec{z}_t$, such as readings from GPS, LiDAR, or cameras.
- These measurements help reduce uncertainty and provide more accurate localization

Basic Math: Reasoning Over Time

Mathematically speaking:

- The term $p(\vec{x}_t \mid \vec{z}_{0:t}, \vec{u}_{0:t})$ represents the Bayesian update for the robot's pose.
- It combines **motion information** $\vec{u}_{0:t}$ (from the control commands) and **sensor measurements** $\vec{z}_{0:t}$ to update the robot's belief about its position over time.

Basic Math: Reasoning Over Time

Conditional probabilities:

- These are conditional probabilities that model how likely a specific robot pose is, given the past motion and current sensor data.
- For example, how likely is the robot to be in a certain location after executing a series of motions and making sensor observations?

Basic Math: Reasoning Over Time

Using Bayes' rule to answer the localization question:

Bayes' rule is applied to compute the **best estimate of the robot's pose**, combining all previous motion information and the latest sensor data.

→ This enables robots to localize themselves accurately over time, even in uncertain environments.

Basic Math: Bayesian Inference

- Initial point: Bayes' theorem

$$P(A | B) = \frac{P(B | A) * P(A)}{P(B)}$$

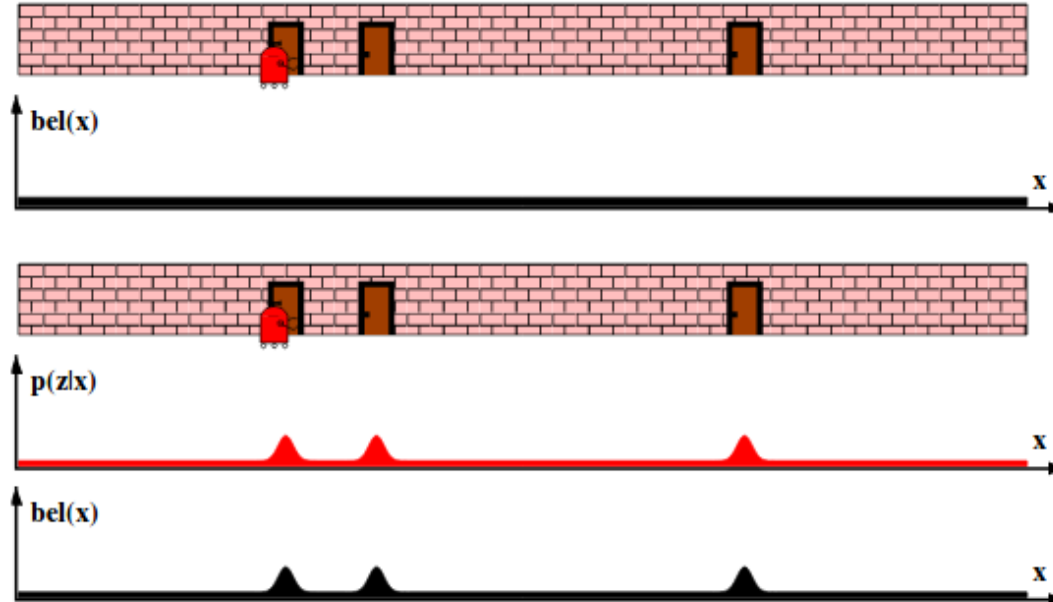
- Then we can inference using measurements
- Example: A door sensor
 - We did n measurements using a “door sensor”: $p(\text{sensor} = T | \text{door} = T)$
 - We know $p(\text{sensor} = T)$ and $p(\text{door} = T)$
 - $p(s) = p(\text{sensor} = \text{True})$ and $p(\neg s) = p(\text{sensor} = \text{False})$

Basic Math: Bayesian Inference

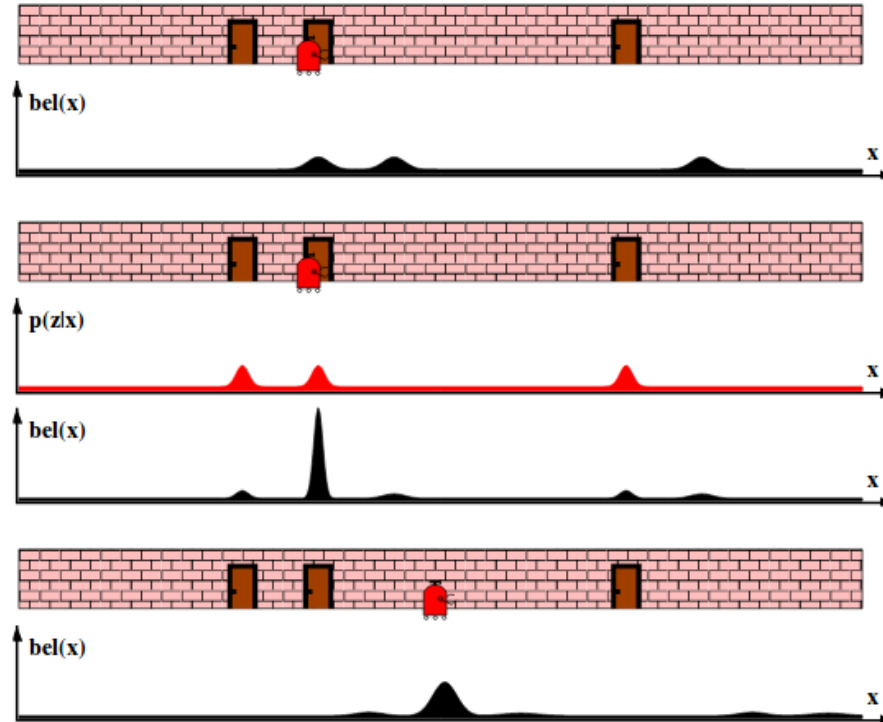
We can calculate $p(d = T \mid s = T)$:

$$\begin{aligned} p(d \mid s) &= \frac{p(s \mid d) * p(d)}{p(s)} = \frac{p(s \mid d) * p(d)}{\sum_{d'} p(s \mid d') p(d')} = \\ &= \frac{p(s \mid d) * p(d)}{p(s \mid d) * p(d) + p(s \mid \neg d) * p(\neg d)} \end{aligned}$$

Basic Math: Bayesian Inference



Basic Math: Bayesian Inference



Basic Math: Bayesian Inference and Conditional Independence

We can now use Bayesian inference for:

- Reasoning over time: $p(\overrightarrow{x_t} \mid \overrightarrow{y_{t=0}}, \dots, \overrightarrow{y_{t=n}})$
- Using conditional independence to simplify equations
- Using the Markov assumption

We can simplify using independent random variables.

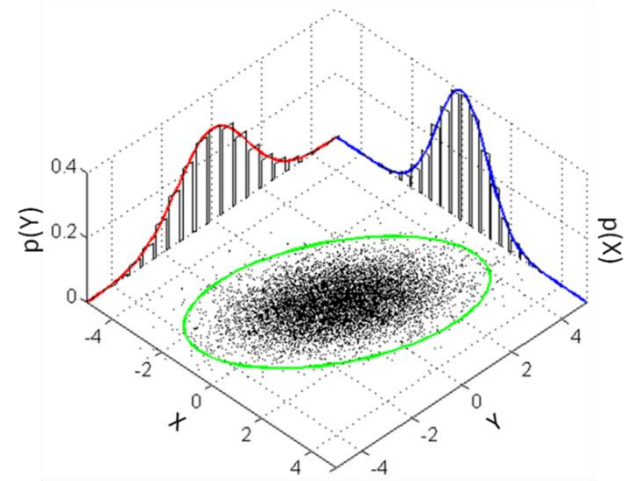
Thus, we can "ignore" variables in equations, variable c carries no information if b is known:

$$p(a \mid b, c) = p(a \mid b)$$

$$p(a, b \mid c) = p(a \mid c) * p(b \mid c)$$

Basic Math: Multivariate Gaussian Distribution

- We must describe random variables
- Simplest approach: Gaussian distribution
- Unimodal distribution: One "peak"
- Moments of distribution: $\vec{\mu}$, Σ



$$p(\vec{x}) = \det(2\pi\Sigma)^{\frac{1}{2}} e^{(-\frac{1}{2}(\vec{x}-\vec{\mu})^T \Sigma^{-1}(\vec{x}-\vec{\mu}))}$$

Probabilistic Robotics: Basics

- We cannot calculate movement (error, noise)
- But we can
 - describe error (e.g.: using gaussian noise)
 - merge information (e.g.: sensor signals)
 - use external knowledge (e.g.: landmarks)
- Content: Merge information
 - Probabilistic robotics (see (Thrun, 2006))
 - Machine Learning (see (Bishop, 2006), (Hastie et al., 2009))
 - AI (see (Russel, 2004), (Cawsey, 1997))
 - Statistical reasoning (see (Perl, 1988))

Probabilistic Robotics: Basics

- Merge all information: $p(\vec{x}_t \mid \vec{x}_{0:t}, \vec{z}_{0:t}, \vec{u}_{0:t})$
- Question: How can we calculate that? Simplifications?
- Physical approach: Effect of $\vec{x}_{t-1}$ on $\vec{x}_t$?
 - Statistical description of $\vec{x}_{t-1}$
 - Movement command $\vec{u}_{0:t}$
- We can simplify following term using conditional independence and Markov assumption

$$p(\vec{x}_t \mid \vec{x}_{0:t-1}, \vec{z}_{0:t-1}, \vec{u}_{1:t}) = p(\vec{x}_t \mid \vec{x}_{t-1}, \vec{u}_t)$$

Basic Math: Markov Assumption

- We want to do localization.
- We assume a "complete state".
 - We can estimate $\vec{x}_t$ using $\vec{x}_{t-1}$ (not $\vec{x}_{0:t-1}$)
 - $\vec{x}_{t-1}$ is then complete
- Note, violations leads to faulty calculations
 - E.g. robot behavior depends on time due to battery state

Probabilistic Robotics: Basics

- We use the Markov assumption, conditional independence and Bayes rule:

$$\begin{aligned} p(\vec{x}_t | \vec{z}_{1:t}, \vec{u}_{1:t}) &= \frac{p(\vec{z}_t | \vec{x}_t, \vec{u}_{1:t}, \vec{z}_{1:t-1}) p(\vec{x}_t | \vec{u}_{1:t}, \vec{z}_{1:t-1})}{p(\vec{z}_t | \vec{z}_{1:t-1}, \vec{u}_{1:t})} = && \text{(Bayes rule)} \\ &= \frac{p(\vec{z}_t | \vec{x}_t) p(\vec{x}_t | \vec{u}_{1:t}, \vec{z}_{1:t-1})}{\alpha^{-1}} = && \text{(cond. independence)} \\ &= \alpha p(\vec{z}_t | \vec{x}_t) \int p(\vec{x}_t | \vec{u}_t, \vec{x}_{t-1}) p(\vec{x}_{t-1} | \vec{z}_{1:t-1}, \vec{u}_{1:t-1}) d\vec{x}_{t-1} \end{aligned} \quad (12)$$

- In the last step, we "add an $\vec{x}_{t-1}$ using integrals".
- This looks complicated, let us analyse the solution step-by-step

Probabilistic Robotics: Basics

The motion model

- An estimation of the recent state using the last state
 - We must use a kinematic model here
 - Based on the last state and a motion command we get a new state

$$p(\vec{x}_t | \vec{u}_t, \vec{x}_{t-1})$$

- An estimation for the previous step (Note, it is a recursive approach):

$$p(\vec{x}_{t-1} | \vec{z}_{1:t-1}, \vec{u}_{1:t-1})$$

Probabilistic Robotics: Basics

The sensor model

$$p(\vec{z}_t | \vec{x}_t)$$

- This evaluates the recent state
- We use a prediction for this evaluation

However, we need to get a pseudo measurement

- We use the motion model and motion commands to calculate the robots new pose
- This is known as the "prediction"
- Using the sensor model, we "correct" the prediction
- The result is known as the "correction"

Probabilistic Robotics: Basics

The Bayes Filter

This results in a two-step approach

- Step 1: Prediction

- We rely on the previous state
- The robot is "moved" using the kinematic model and the command
- We solve the integral, we get the prediction

The prediction (the conditional probability) is shortened to $\overline{bel}(\vec{x}_t)$

Probabilistic Robotics: Basics

The Bayes Filter

- Step 1: Prediction
- Step 2: Correction
 - Evaluation of prediction
 - Correction of prediction using a measurement
 - The correction (the conditional probability) is shortened using $bel(\vec{x}_t)$

Probabilistic Robotics: Basics

- Merging $p(\vec{x}_t | \vec{x}_{t-1}, \vec{u}_t)$ and $p(\vec{z}_t | \vec{x}_t)$
- Two-stage algorithm: Movement prediction and correction
- Approach:

Iterative algorithm: $\vec{x}_t \rightarrow \vec{x}_{t+1}$

Calculation using $p(\vec{x}_t | \vec{x}_{t-1}, \vec{u}_t)$

Correction using $p(\vec{z}_t | \vec{x}_t)$

Theoretical approach: Bayes Filter

```
1:   Algorithm Bayes_filter( $bel(x_{t-1}), u_t, z_t$ ):  
2:     for all  $x_t$  do  
3:        $\overline{bel}(x_t) = \int p(x_t | u_t, x_{t-1}) bel(x_{t-1}) dx$   
4:        $bel(x_t) = \eta p(z_t | x_t) \overline{bel}(x_t)$   
5:     endfor  
6:     return  $bel(x_t)$ 
```

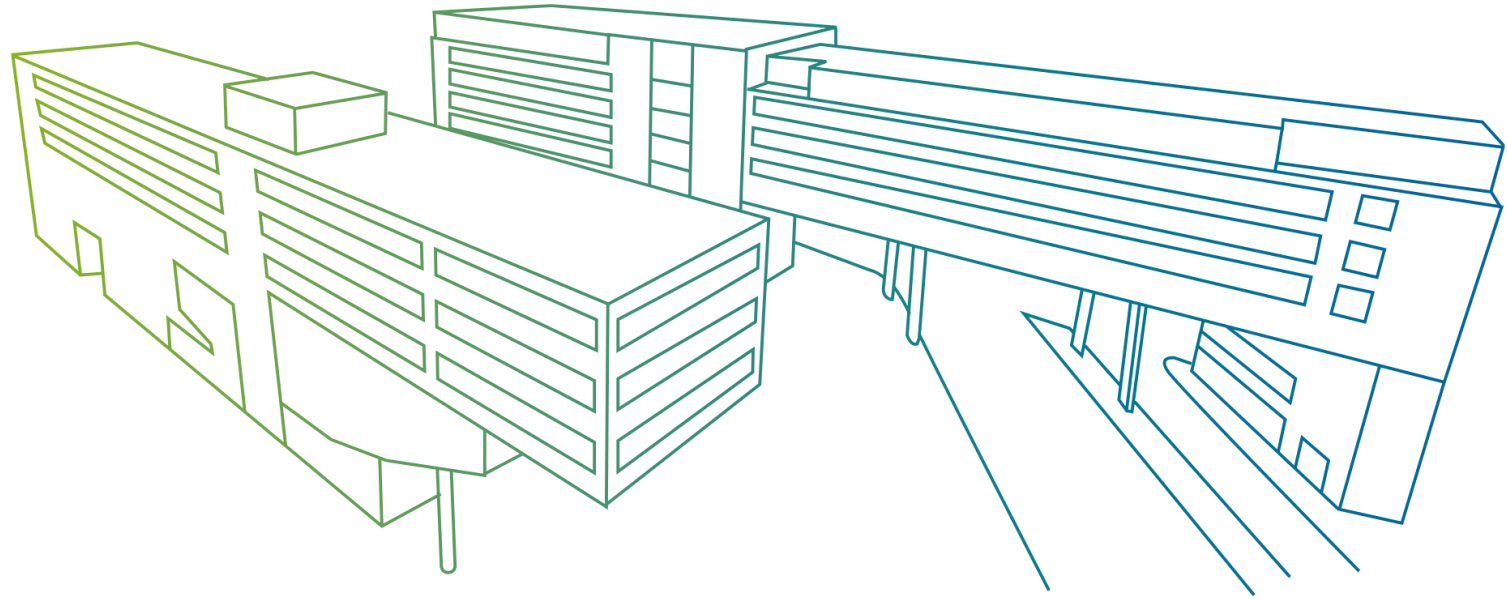
Figure: Bayes Filter Localization (see page 24 in (Thrun, 2006))

Next Steps

- Kalman Filter:
 - Estimating states in linear Gaussian systems.
 - Predict and update steps for state estimation.
- Extended Kalman Filter (EKF):
 - Handling non-linear systems with Taylor approximation.
 - EKF for localization and sensor fusion.

Next Steps

- Particle Filter
 - Monte Carlo methods for non-Gaussian, non-linear systems
 - Particle-based state estimation and resampling
- Motion and Sensor Models
 - How robots model their movement with uncertainty
 - Sensor models: interpreting data from noisy sensors (e.g., LiDAR, cameras)



Probabilistic Robotics

Basic Introduction
Questions?